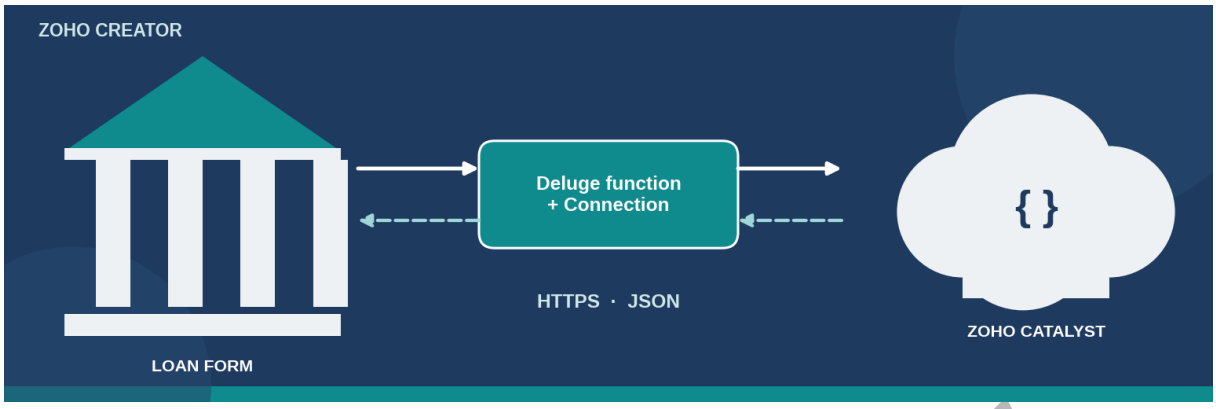


EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst and Custom Functions

Bank Loan Application Form — Priority Scoring Extension Use Case

Submitted by	**Sharmila G.**
Register Number	241401097
Department	Computer Science and Business Systems
Document Type	Laboratory Experiment Report

241401097

1. Aim

The aim of this experiment is to examine how a loan application system built on Zoho Creator can be extended beyond the limits of low-code configuration. Two layers of custom logic are combined for this purpose: a reusable Deluge custom function that lives inside Creator, and a serverless function written in Node.js and hosted on Zoho Catalyst. Together they make it possible to add requirements that are too intricate for point-and-click workflow actions, without rebuilding the application.

A priority-scoring extension for a Bank Loan Application Form is used as the worked example. When an applicant submits the form, the loan tenure, interest rate and loan purpose are passed to an external pro-code function, which returns a review priority of Normal, Medium or High. The example shows how Creator and an external function communicate over a secure HTTPS connection using JSON, when each extension option is appropriate, and which security, error-handling and maintenance precautions a real implementation would need.

2. Procedure

1. **Identify the extension need.** The loan application form and its workflow were reviewed, and a requirement was selected that built-in actions handle poorly: assigning each application a Normal, Medium or High review priority from its tenure, interest rate and purpose.
2. **Compare the extension options.** Built-in workflow actions, Deluge custom functions, Creator Custom APIs and Catalyst serverless functions were compared on skill level and suitable use (Section 3.2).
3. **Design the architecture.** The Creator side (form, workflow, custom function and connection) and the Catalyst side (Advanced I/O function with optional storage or external API) were laid out in a single diagram (Section 3.1).
4. **Prepare the Deluge custom function.** An illustrative reusable function was written to assemble a JSON payload and call the Catalyst endpoint through a connection (Section 3.3).
5. **Prepare the Node.js function.** An illustrative Advanced I/O function was written to validate the input, apply example weights and return the priority as JSON (Section 3.4).
6. **Design the execution flow and decision guide.** The end-to-end sequence, including a separate failure branch, was drawn as a flowchart (Section 3.5), and a guide for selecting the extension type was prepared (Section 3.6).
7. **Identify precautions.** Security, error handling, platform limits and maintenance points for this style of integration were recorded (Section 3.7).

Note: *This report explores an approach. All code is illustrative and was not deployed or executed. The endpoint, connection, form and field names (including Priority_Level) are placeholders, and syntax should be verified against the current Zoho documentation.*

2414O1097

3. Output

3.1 Architecture of the Pro-Code Extension

In this design Zoho Creator remains the home of the loan application: it holds the form, the stored records and the workflow that reacts to a submission. Zoho Catalyst contributes a serverless environment where code that is awkward to express in Creator can run independently. The Deluge custom function and the secure connection form the bridge, so Creator can hand data to Catalyst and receive a result without exposing credentials in the script.

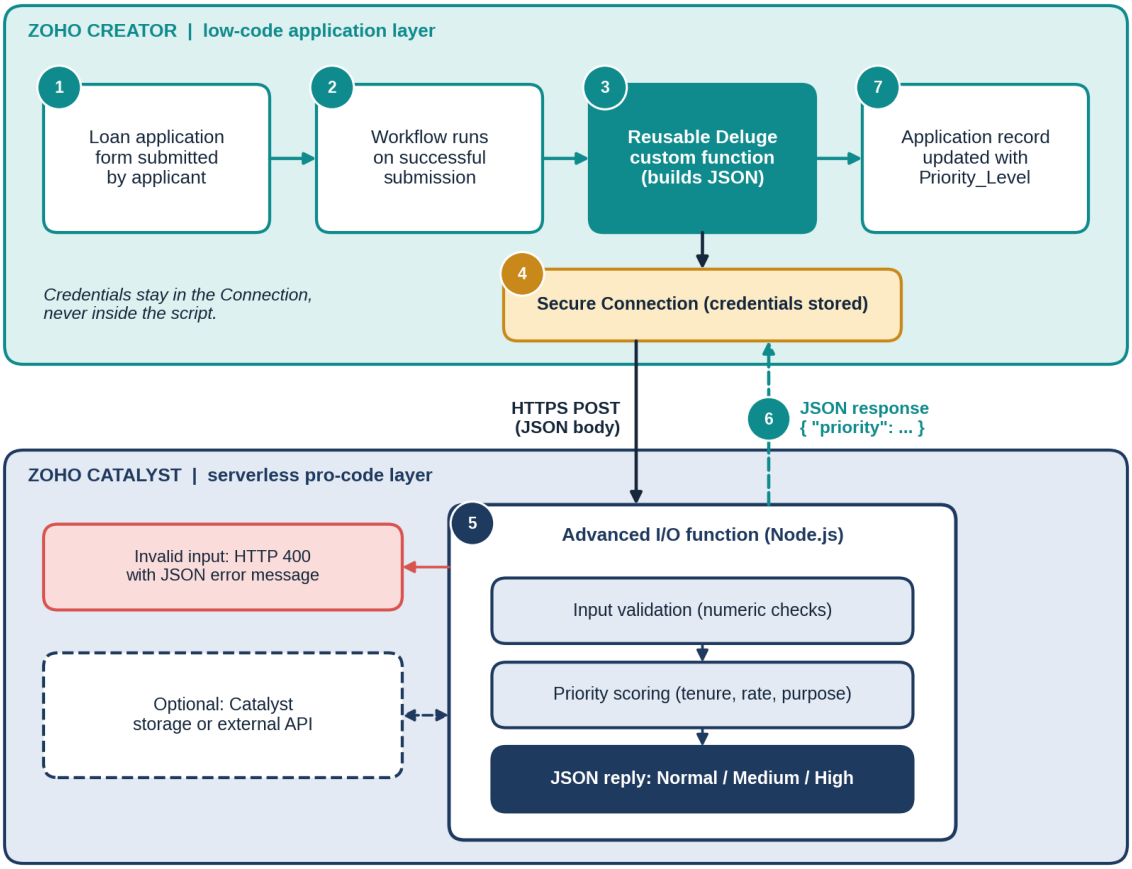


Figure 1: *Architecture of the Zoho Creator and Zoho Catalyst pro-code extension (illustrative).*

The interaction begins when an applicant submits the loan form (1). After the record is saved, a workflow triggered by the successful submission (2) calls the reusable Deluge custom function (3), which collects the loan tenure, interest rate and loan purpose into a JSON payload. The function sends the payload through a configured connection (4), which keeps the authentication details outside the code, as an HTTPS POST request to the Catalyst Advanced I/O function (5). Inside Catalyst, the Node.js code validates the numeric input and computes the priority score. If the input is invalid, it replies with HTTP 400 and an error message instead. The function may optionally use Catalyst storage or an external API for further processing. A JSON response containing the priority returns to Creator (6), and the workflow stores it in the Priority_Level field of the application record (7).

3.2 Comparison of Extension Options

Zoho Creator offers several ways to add behaviour, and they differ in the skill they demand and the complexity they can absorb. Simple automation is best handled by configuration, while

logic that is shared across forms benefits from reusable code, and requirements involving libraries or outside systems justify a separate runtime. Choosing the lightest option that satisfies the requirement keeps the application easier to build and maintain.

Table 1: *Comparison of Zoho Creator extension options.*

Extension Option	Description	Skill Required	Suitable Use Case
Built-in workflow actions	Ready-made actions such as sending notifications, adding records or updating fields, configured in the builder.	Low (no code)	Standard, straightforward automation.
Deluge custom functions	Reusable functions written in Deluge and called from workflows, schedules or other functions.	Low to medium	Business rules shared across several workflows inside Creator.
Creator Custom APIs	REST endpoints exposed by the Creator application so that other systems can call into it.	Medium	Allowing external systems to read or write Creator data.
Zoho Catalyst serverless functions	Functions in Node.js, Java or Python that run outside Creator and are reached over HTTPS.	Medium to high (pro-code)	Complex logic, external libraries, heavy processing or outside systems.

The table orders the options by increasing capability and effort. Workflow actions need no programming but cover only predefined behaviour. Deluge custom functions add reusable scripted logic while staying within Creator. A Custom API points in the opposite direction: it lets external systems call Creator rather than extending Creator with new processing. Catalyst functions offer the greatest freedom, including libraries and external services, at the cost of additional code, deployment and a network call. For the priority-scoring example, a Deluge function bridges to a Catalyst function to demonstrate the pro-code option.

3.3 Deluge Custom Function

A custom function packages a piece of business logic once so that any workflow can call it with different inputs. In this experiment the function hides the details of building the request and calling Catalyst, so the form workflow needs only a single line to obtain a priority value. If the endpoint or payload later changes, only the function needs to be updated.

24140

```
// Standalone function (illustrative): computeLoanPriority
string computeLoanPriority(int tenureMonths, decimal interestRate,
                           string loanPurpose)
{
    // Default retained if the external call fails
    priorityLevel = "Normal";
    // 1. Construct the JSON payload
    payload = Map();
    payload.put("tenure months", tenureMonths);
    payload.put("interest rate", interestRate);
    payload.put("loan purpose", loanPurpose);
    try
    {
        // 2. Call the Catalyst endpoint through a configured connection
        response = invokeurl
        [
            url : "<CATALYST ADVANCED IO ENDPOINT>/score"
            type : POST
            parameters : payload.toString()
            headers : {"Content-Type" : "application/json"}
            connection : "catalyst_connection"
        ];

        // 3. Read the calculated priority from the JSON response
        if(response.get("priority") != null)
        {
            priorityLevel = response.get("priority");
        }
    }
    catch (e)
    {
        info "Priority service unavailable: " + e;
    }

    // 4. Return the result to the calling workflow
    return priorityLevel;
}
// Workflow: Loan_Application form > Add > On success (placeholder names)
level = thisapp.computeLoanPriority(input.Tenure_Months,
                                    input.Interest_Rate, input.Loan_Purpose);
Loan Application[ID == input.ID].Priority Level = level;
```

Figure 2: *Illustrative Deluge custom function and workflow call (not executed).*

The function accepts the tenure in months, the interest rate and the loan purpose, places them in a Map and converts it to a JSON string. It then uses invokeurl to send an HTTPS POST request through the named connection, reads the priority key from the response and returns it. A default value of Normal is set first and a try-catch block is used, so a failed call leaves the default in place instead of stopping the workflow. The workflow then updates the record using the returned value. The endpoint placeholder <CATALYST_ADVANCED_IO_ENDPOINT>, the connection name catalyst_connection, the form name Loan_Application and the field names Tenure_Months, Interest_Rate, Loan_Purpose and Priority_Level are placeholders. No claim is made that this code has been run or deployed.

3.4 Catalyst Advanced I/O Function in Node.js

A serverless function lets the scoring logic run outside Creator, where it can use the full capabilities of a general-purpose language. The Advanced I/O function exposes an HTTP endpoint, reads the JSON body sent by the Deluge function, applies the rules and answers with JSON. Because it has no fixed server to manage, it fits short, request-driven tasks of this kind.

```
"use strict";
const express = require("express");
const app = express();
app.use(express.json());

// Illustrative weights only - NOT actual bank lending rules
const PURPOSE WEIGHTS = new Map([
  ["business", 3], ["home", 2], ["vehicle", 1], ["personal", 1], ["education", 0]
]);

function toNumber(value) {
  if (typeof value === "string" && value.trim() !== "") value = Number(value);
  return typeof value === "number" && Number.isFinite(value) ? value : null;
}
const tenurePoints = m => (m >= 120 ? 3 : m >= 60 ? 2 : m >= 24 ? 1 : 0);
const ratePoints   = r => (r >= 14 ? 3 : r >= 10 ? 2 : r >= 7 ? 1 : 0);

app.post("/score", (req, res) => {
  const { tenure_months, interest_rate, loan_purpose } = req.body || {};
  const tenure = toNumber(tenure months);
  const rate = toNumber(interest_rate);

  // Validate numeric input
  if (tenure === null || rate === null || tenure <= 0 || rate < 0) {
    return res.status(400).json({
      error: "tenure_months and interest_rate must be valid numbers"
    });
  }

  // Evaluate the example weights
  const purposeKey = String(loan purpose || "").trim().toLowerCase();
  const score = tenurePoints(tenure) + ratePoints(rate)
              + (PURPOSE WEIGHTS.get(purposeKey) ?? 0);

  // Illustrative thresholds
  const priority = score >= 6 ? "High" : score >= 3 ? "Medium" : "Normal";
  return res.status(200).json({ priority, score });
});

module.exports = app;
```

Figure 3: *Illustrative Node.js Advanced I/O function for priority scoring (not executed).*

The function reads tenure_months, interest_rate and loan_purpose from the request body and passes the numeric fields through a helper that accepts only finite numbers. If either value is missing, non-numeric, or out of range, it returns HTTP 400 with a JSON error message. Otherwise it adds points for tenure, interest rate and loan purpose, and maps the total to Normal, Medium or High, returning the result with HTTP 200. Importantly, the weights, thresholds and purpose categories shown here are illustrative examples only and are not actual bank lending rules; a real institution's credit and risk team would have to define and approve them.

3.5 Execution Flow of the Extended Workflow

The end-to-end process runs from the applicant's submission to the updated record. The sequence is triggered automatically by the workflow, passes through the custom function to Catalyst, and returns a result that is checked before it is stored. Designing the failure path alongside the normal path ensures that a problem with the external service never prevents a loan application from being captured.

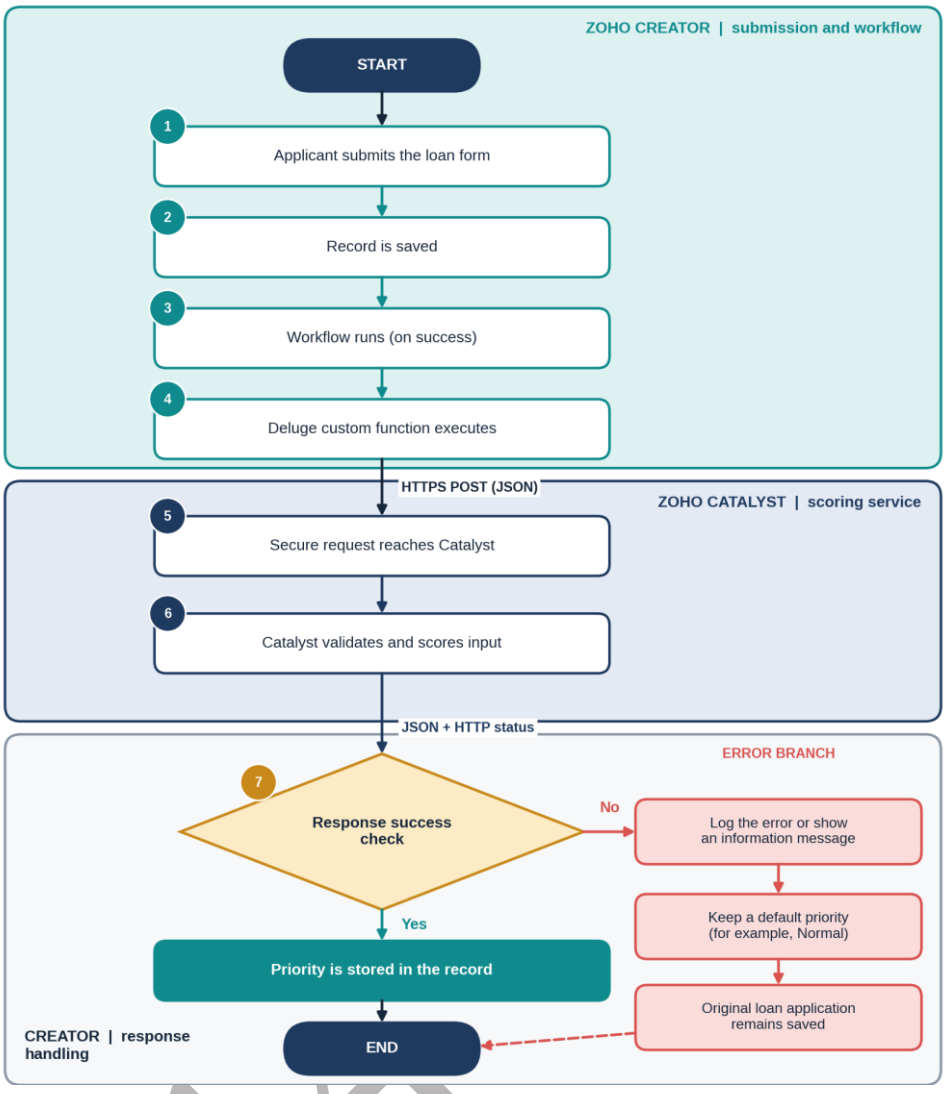


Figure 4: *Execution flow of the extended workflow, including the error branch.*

After the applicant submits the form, the record is saved and the workflow runs, calling the Deluge custom function. The function sends a secure request to Catalyst, where the input is validated and scored. Creator then performs a response success check. On success, the priority is stored in the record and the process ends. If the response fails, a separate branch logs the error or shows an information message, keeps a suitable default priority and leaves the original loan application saved, so no application data is lost because of the extension.

3.6 Decision Guide for Choosing the Extension Type

Developers should select an extension method according to how complex the requirement is and whether the logic must be reused. A sensible approach is to start with the simplest tool and move to a heavier one only when the simpler tool cannot meet the need, since each step adds code to build, test and maintain.

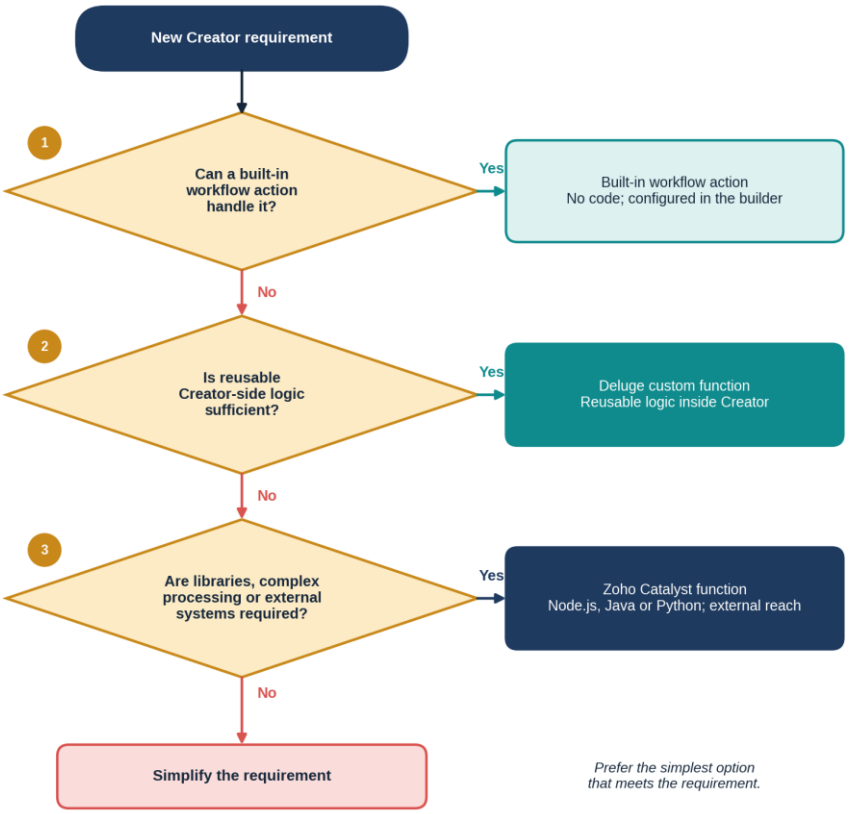


Figure 5: *Decision guide for choosing the extension type.*

The guide asks three questions in order. If a built-in workflow action can handle the requirement, it should be used. If not, and reusable Creator-side logic is sufficient, a Deluge custom function is appropriate. If external libraries, complex processing or external systems are required, a Zoho Catalyst function is justified. When none of these applies, the requirement itself should be simplified rather than adding unnecessary complexity.

3.7 Benefits and Precautions

Extending Zoho Creator with pro-code functions widens what the application can do, but it also introduces additional parts that must be secured, monitored and maintained. The table below summarises the main advantages together with the matching precautions.

24140

Table 2: *Benefits and precautions of pro-code extension.*

Aspect	Benefit	Precaution
Flexibility	Any logic or library available in Node.js, Java or Python can be used.	More code to write, test and maintain than low-code.
Reusability	A custom function is written once and called from many workflows.	Keep function inputs and outputs stable so callers do not break.
Separation of processing	Heavy work runs outside Creator, keeping the application light.	Adds a network call, so set timeouts and handle failures.
Security	Credentials are held in a connection rather than in the script.	Keep keys out of code, restrict access and validate all input.
Execution limits and cost	The platform provides serverless scaling without managing servers.	Check current execution limits and pricing in the documentation.
Error handling and maintenance	Defaults and logging allow the workflow to continue safely.	Log failures, monitor the function and version changes carefully.

The principal benefit is flexibility: logic that cannot be expressed through workflow actions can still be added. Reusability reduces duplication, because the same Deluge function can serve several forms. Separating processing keeps Creator responsive and allows scoring rules to change independently. These gains depend on careful practice. Credentials should remain in connections, inputs must be validated on the Catalyst side, and execution limits and pricing should be checked in current Zoho documentation. Because every call crosses a network, timeouts and a safe default are essential, and failures should be logged so the integration can be maintained over time.

4. Final Result

This experiment explored how a low-code Zoho Creator application can be extended with pro-code logic in two steps. The architecture connects the Creator loan form and workflow to a Zoho Catalyst Advanced I/O function through a reusable Deluge custom function and a secure connection, exchanging data as JSON over HTTPS.

The reusable Deluge custom function packages the request logic once so any workflow can obtain a priority with a single call. The Catalyst Node.js function validates the input, applies example weights and returns a Normal, Medium or High priority, or an error response for invalid data. The priority-scoring workflow stores the returned value in the application record, and its failure branch keeps a default value so the original application is never lost.

The decision guide showed that built-in workflow actions and custom functions meet most needs, with Catalyst reserved for complex logic, libraries or outside systems. Security, error handling and maintenance precautions, including protected credentials, input validation, timeouts and monitoring of limits and costs, were identified for a real implementation.

The code, scoring weights and integration described here are illustrative. They have not been deployed or tested, and no results of a live run are claimed.

24140109